WINTER SEMESTER 2025-26 ASSESSMENT - 1

Registration NO: 23MIS0141

Name: KANISH KUMAR S

School: School of Computer Science Engineering and Information Systems (SCORE) Course Code: ISWE406P Course Title: Agile Development Process and Devops

Lab Faculty: BALA GANESH

In case of blur in screenshots kindly zoom in

Case Study 1:

New Project Version Control Setup Scenario: You are assigned to develop a Library Management System. Before coding, your team decides to enable version control.

Tasks: - Create a project folder and open it in VS Code -

Initialize a Git repository -

git init

```
~/Public/A$git init
hint: Using 'master' as the name for the initial branch. This default branch name
hint: is subject to change. To configure the initial branch name to use in all
hint: of your new repositories, which will suppress this warning, call:
hint:
hint:   git config --global init.defaultBranch <name>
hint:
hint: Names commonly chosen instead of 'master' are 'main', 'trunk' and
hint: 'development'. The just-created branch can be renamed via this command:
hint:
hint:   git branch -m <name>
Initialized empty Git repository in /home/kanish-kumar/Public/A/.git/
~/Public/A$
```

Create a Python file and track it using Git -

touch login.py

```
~/Public/A$touch  login.py
~/Public/A$ls
login.py
~/Public/A$
~/Public/A$
```

login.py M

login.py > ...

```python
a=input("username:")
db={"kanish kumar":"2005","tharun":"2006","vinay":"2003","charan":"333","velluri":"234"}
while(a not in db.keys()):
    print("invalid  user")
    a=input("username:")
b=input("password:")
while not(db[a]==b):
    print("wrong password")
    b=input("password:")
```

git add login.py

```
~/Public/A$git add login.py
~/Public/A$
```

Commit the initial version

git commit –m “first commit”

```
~/Public/A$git commit -m "first commit"
[master (root-commit) ddc0409] first commit
 1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 login.py
~/Public/A$
```

Case Study 2:

Tracking Daily Code Changes Scenario: You are working on a Student Registration Module. You make small updates daily and need to record them.

Tasks: -

Modify an existing file -

```
login.py M
login.py > ...
a=input("username:")
db={"kanish kumar":"2005","tharun":"2005","vinay":"2003","charan":"333","velluri":"234"}
while(a not in db.keys()):
    print("invalid  user")
    a=input("username:")
b=input("password:")
while not(db[a]==b):
    print("wrong password")
    b=input("password:")
print("welcome",end=" "+a+"\n")
```

Check file differences -

git diff login.py

```
~/Public/A$git diff login.py
diff --git a/login.py b/login.py
index 654b9ef..89887ab 100644
--- a/login.py
+++ b/login.py
@@ -1,9 +1,10 @@
 a=input("username:")
-db={"kanish kumar":"2005","tharun":"2006","vinay":"2003","charan":"333","velluri":"234"}
+db={"kanish kumar":"2005","tharun":"2005","vinay":"2003","charan":"333","velluri":"234"}
 while(a not in db.keys()):
     print("invalid  user")
     a=input("username:")
 b=input("password:")
 while not(db[a]==b):
     print("wrong password")
-    b=input("password:")
\ No newline at end of file
+    b=input("password:")
+print("welcome",end=" "+a+"\n")
~/Public/A$
```

Stage and commit changes -

git add .

git commit –m “second-commit”

```
~/Public/A$git commit -m "second-commit"
[master 52a9bd8] second-commit
 1 file changed, 10 insertions(+)
~/Public/A$
~/Public/A$
```

View commit history

git log

```
~/Public/A$git log
commit 52a9bd81e60ac576c43ef32a47b50226e77cbcd7 (HEAD -> master)
Author: Kanish-Kumar-S <kanishkumar.s2023@vitstudent.ac.in>
Date:   Sun Jan 18 11:14:30 2026 +0530

    second-commit

commit ddc0409ab6a9597ca31253a8812f39ca4b0fd409
Author: Kanish-Kumar-S <kanishkumar.s2023@vitstudent.ac.in>
Date:   Sun Jan 18 09:41:49 2026 +0530

    first commit
~/Public/A$
```

Case Study 3

: Feature Development Using Branches Scenario:

A new Login feature must be developed without disturbing main code.

Tasks:

Create a new branch -

git branch -c "new-login-feature"

git checkout new-login-feature

```
~/Public/A$git branch -c "new-login-feature"
~/Public/A$git branch
* master
  new-login-feature
~/Public/A$git checkout new-login-feature
Switched to branch 'new-login-feature'
~/Public/A$
```

Add login-related code -

```python
from datetime import date
a=input("username:")
db={"kanish kumar":"2005","tharun":"2005","vinay":"2003","charan":"333","velluri":"234"}
while(a not in db.keys()):
    print("invalid  user")
    a=input("username:")
b=input("password:")
while not(db[a]==b):
    print("wrong password")
    b=input("password:")

print("welcome ",end=" "+a+"\n")
print("date:")
print(date.today())
```

Commit changes -

git add .

git commit -m “new login-1”

```
~/Public/A$git add .
~/Public/A$git commit -m "new-login-1"
[new-login-feature dc62f5b] new-login-1
 1 file changed, 5 insertions(+), 2 deletions(-)
```

Switch between branches

git switch master

git branch

git switch new-login-feature

```
~/Public/A$git commit -m "new-login-1"
[new-login-feature dc62f5b] new-login-1
 1 file changed, 5 insertions(+), 2 deletions(-)
~/Public/A$git switch master
Switched to branch 'master'
~/Public/A$git branch
* master
  new-login-feature
~/Public/A$git switch new-login-feature
Switched to branch 'new-login-feature'
~/Public/A$
```

Case Study 4:

Merging Feature into Main Code Scenario: The login feature is tested and approved.

Tasks: -

Switch to main branch -

git switch master

```
~/Public/A$git switch new-login-feature
Switched to branch 'new-login-feature'
~/Public/A$git switch master
Switched to branch 'master'
~/Public/A$
```

Merge feature branch -

git merge new-login-feature

git switch master

```
~/Public/A$git switch master
Switched to branch 'master'
~/Public/A$git merge new-login-feature
Updating 52a9bd8..dc62f5b
Fast-forward
 login.py | 7 ++++++-
 1 file changed, 6 insertions(+), 1 deletion(-)
~/Public/A$
```

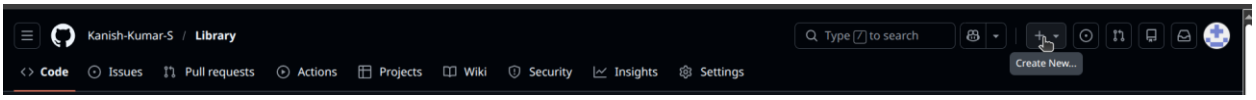
Verify merged content

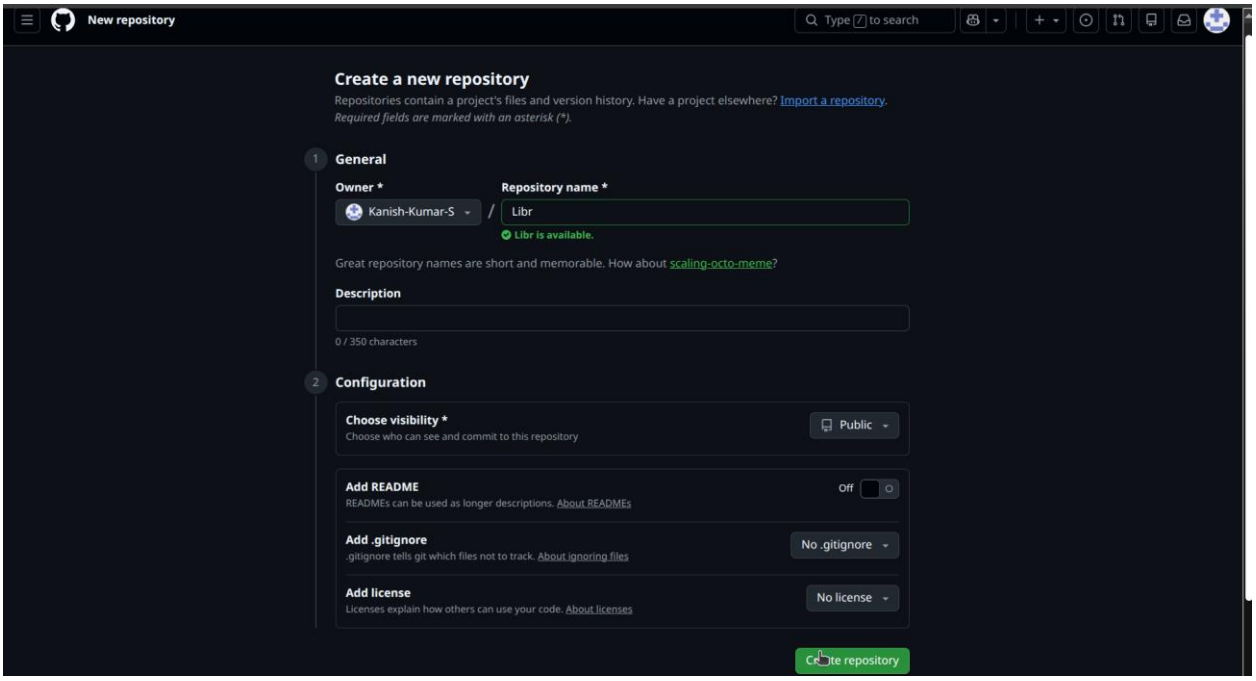
```
~/Public/A$git branch
 * master
   new-login-feature
~/Public/A$cat login.py
from datetime import date
a=input("username:")
db={"kanish kumar":"2005","tharun":"2005","vinay":"2003","charan":"333","velluri":"234"}
while(a not in db.keys()):
    print("invalid  user")
    a=input("username:")
b=input("password:")
while not(db[a]==b):
    print("wrong password")
    b=input("password:")

print("welcome ",end=" "+a+"\n")
print("date:")
print(date.today())
~/Public/A$
```

Case Study 5: Working with Remote Repository Scenario:

The project must be shared using GitHub.

Tasks: - Create remote repository -





Add remote origin -

git remote add origin https://github.com/Kanish-Kumar-S/Library

```
new-login-feature
~/Public/A$git remote add origin https://github.com/Kanish-Kumar-S/Library
~/Public/A$
~/Public/A$
~/Public/A$
~/Public/A$
~/Public/A$
~/Public/A$
```

Push code to GitHub

git push origin master

```
~/Public/A$git push origin master
Enumerating objects: 12, done.
Counting objects: 100% (12/12), done.
Delta compression using up to 16 threads
Compressing objects: 100% (7/7), done.
Writing objects: 100% (12/12), 1.12 KiB | 1.13 MiB/s, done.
Total 12 (delta 3), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (3/3), done.
To https://github.com/Kanish-Kumar-S/Library
 * [new branch]      master -> master
~/Public/A$
```

Case Study 6: Undoing Mistakes Scenario: A file is modified incorrectly.

Tasks: -

View file changes -

git diff login.py

git status

```python
from datetime import date
a=input("username:")
db={"kanish kumar":"2005","tharun":"2005","vinay":"2003","charan":"333","velluri":"234"}
while(a not in db.keys()):
    print("invalid  user")
    a=input("username:")
b=input("password:")
while not(db[a]==b):
    print("wrong password")
    b=input("password:")

print("welcome ",end=" "+a+"\n")
print("date:")
print(date.today())
vsdnv
sdvajjlnldsa
vsadlinvlsnvl
vsdajvnsadnvl
```

```
~/Public/A$git diff login.py
~/Public/A$git diff login.py
diff --git a/login.py b/login.py
index f2c290e..89df173 100644
--- a/login.py
+++ b/login.py
@@ -11,4 +11,5 @@ while not(db[a]==b):

 print("welcome ",end=" "+a+"\n")
 print("date:")
-print(date.today())
\ No newline at end of file
+print(date.today())
+svlnvadslvnlsav
\ No newline at end of file
~/Public/A$
```

```
~/Public/A$git status
On branch master
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   login.py
```

Discard uncommitted changes

git restore --staged login.py

```
~/Public/A$git restore login.py
~/Public/A$
~/Public/A$
```

Case Study 7: Handling Merge Conflicts

Scenario: Two developers edited the same file.

Tasks:

- Merge branches

```
Resolving deltas: 100% (3/3), done.
PS C:\Users\USER\Downloads\New folder (3)> git branch
PS C:\Users\USER\Downloads\New folder (3)> cd Library
>>
PS C:\Users\USER\Downloads\New folder (3)\Library> git branch
* tharun
PS C:\Users\USER\Downloads\New folder (3)\Library>
```

//accidently some commands were not captured in the screenshot

//those are entered below

git switch master

git merge tharun

```
PS C:\Users\USER\Downloads\New folder (3)> ls


    Directory: C:\Users\USER\Downloads\New folder (3)


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-----        18-01-2026     21:36                Library


PS C:\Users\USER\Downloads\New folder (3)> cd Library
PS C:\Users\USER\Downloads\New folder (3)\Library> git add .
PS C:\Users\USER\Downloads\New folder (3)\Library> git commit -m tharuncommit1
Author identity unknown

*** Please tell me who you are.

Run

  git config --global user.email "you@example.com"
  git config --global user.name "Your Name"

to set your account's default identity.
Omit --global to set the identity only in this repository.

fatal: unable to auto-detect email address (got 'USER@LAPTOP-GO174129.(none)')
PS C:\Users\USER\Downloads\New folder (3)\Library> git config --local user.name Tharun-26122005
PS C:\Users\USER\Downloads\New folder (3)\Library> git config --local user.email sreedharan.tharun2023@vitstudent.ac.in
PS C:\Users\USER\Downloads\New folder (3)\Library> git commit -m tharuncommit1
[tharun 8610a24] tharuncommit1
 1 file changed, 2 insertions(+), 2 deletions(-)
PS C:\Users\USER\Downloads\New folder (3)\Library>
```

```
PS C:\Users\USER\Downloads\New folder (3)\Library> git push origin tharun
info: please complete authentication in your browser...
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 16 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 303 bytes | 303.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/Kanish-Kumar-S/Library.git
   dc62f5b..8610a24  tharun -> tharun
PS C:\Users\USER\Downloads\New folder (3)\Library>
```

```
PS C:\Users\USER\Downloads\New folder (3)\Library> git log
commit 8610a248c60b863a6065735155886118 64cb47d5 (HEAD -> tharun)
Author: Tharun-26122005 <sreedharan.tharun2023@vitstudent.ac.in>
Date:   Sun Jan 18 21:55:14 2026 +0530

    tharuncommit1

commit dc62f5ba293b11233bfe35812b388d228b7552a7 (origin/tharun, origin/master, origin/HEAD)
Author: Kanish-Kumar-S <kanishkumar.s2023@vitstudent.ac.in>
Date:   Sun Jan 18 11:56:49 2026 +0530

    new-login-1

commit b6422a6389f79f175a5687c1ead206bfc1a5160b
Author: Kanish-Kumar-S <kanishkumar.s2023@vitstudent.ac.in>
Date:   Sun Jan 18 11:50:57 2026 +0530

    new-login-1

commit 52a9bd81e60ac576c43ef32a47b50226e77cbcd7
Author: Kanish-Kumar-S <kanishkumar.s2023@vitstudent.ac.in>
Date:   Sun Jan 18 11:14:30 2026 +0530

    second-commit

commit ddc0409ab6a9597ca31253a8812f39ca4b0fd409
Author: Kanish-Kumar-S <kanishkumar.s2023@vitstudent.ac.in>
Date:   Sun Jan 18 09:41:49 2026 +0530

    first commit
PS C:\Users\USER\Downloads\New folder (3)\Library>
```

Case Study 8: Ignoring Unwanted Files Scenario: Log and temporary files should not be tracked.

Tasks: -

Create .gitignore file

touch .gitignore (works only linux/unix based machines)

```
~/Public/A$touch .gitignore
~/Public/A$git add .
~/Public/A$git status
On branch master
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        new file:   .gitignore
        modified:   login.py

~/Public/A$git commit -m "gitignore added"
[master e245bdc] gitignore added
 2 files changed, 1 insertion(+), 1 deletion(-)
 create mode 100644 .gitignore
~/Public/A$
```

Add ignore patterns

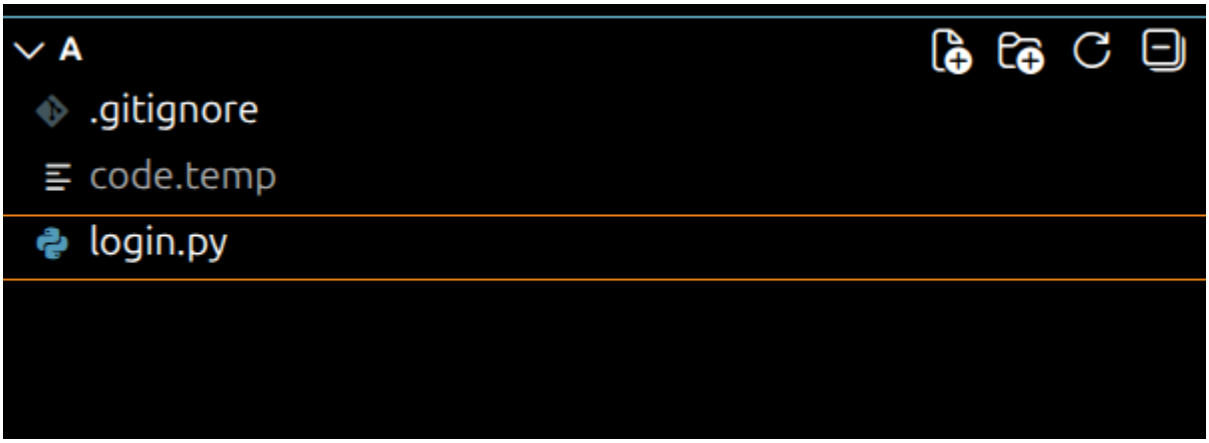
git add .

git status

```
~/Public/A$touch .gitignore
~/Public/A$git add .
~/Public/A$git status
On branch master
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        new file:   .gitignore
        modified:   login.py

~/Public/A$git commit -m "gitignore added"
[master e245bdc] gitignore added
 2 files changed, 1 insertion(+), 1 deletion(-)
 create mode 100644 .gitignore
~/Public/A$
```

```
.gitignore
1  kk.txt
2  *.temp
```

WE Observe that code.temp is ignored while staging



```
~/Public/A$git commit -m "ignore pattern added"
[master a0f0c77] ignore pattern added
 1 file changed, 2 insertions(+)
~/Public/A$git add .
~/Public/A$git status
On branch master
nothing to commit, working tree clean
~/Public/A$git status
On branch master
nothing to commit, working tree clean
~/Public/A$
```

Case Study 9:

Collaborating with Team Scenario:

A teammate pushed new changes. Tasks: -

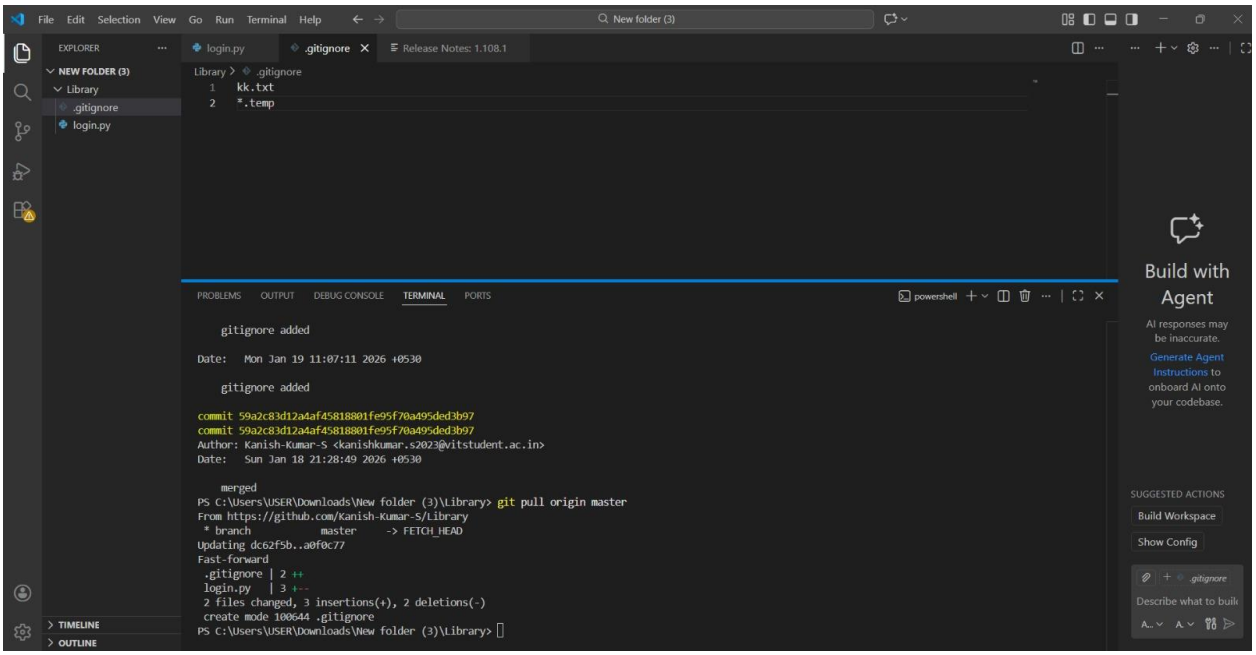
Fetch latest changes -

git fetch origin master

```
fatal: you must specify path(s) to restore
PS C:\Users\USER\Downloads\New folder (3)\Library> git fetch origin master
remote: Enumerating objects: 12, done.
remote: Counting objects: 100% (12/12), done.
remote: Compressing objects: 100% (5/5), done.
remote: Total 10 (delta 2), reused 10 (delta 2), pack-reused 0 (from 0)
Unpacking objects: 100% (10/10), 830 bytes | 23.00 KiB/s, done.
From https://github.com/Kanish-Kumar-S/Library
 * branch            master     -> FETCH_HEAD
   dc62f5b..a0f0c77  master     -> origin/master
PS C:\Users\USER\Downloads\New folder (3)\Library> git log master..origin/master
commit a0f0c77e3770e6749c8287428fdd69075a91acfd (origin/master, origin/HEAD)
Author: Kanish-Kumar-S <kanishkumar.s2023@vitstudent.ac.in>
Date:   Mon Jan 19 11:13:40 2026 +0530

    ignore pattern added

commit e245bdc3d764003e116b52805e065bb0bb0c6701
Author: Kanish-Kumar-S <kanishkumar.s2023@vitstudent.ac.in>
Date:   Mon Jan 19 11:07:11 2026 +0530

    gitignore added

commit 59a2c83d12a4af45818801fe95f70a495ded3b97
Author: Kanish-Kumar-S <kanishkumar.s2023@vitstudent.ac.in>
Date:   Sun Jan 18 21:28:49 2026 +0530

    merged
```

Pull updates -

git pull origin



Resolve conflicts if any

No conflicts

Case Study 10:

Final Project Release Scenario: Project is ready for final submission.

Tasks: -

Review commit history -

Push final code -

Prepare release version

```
~/Public/A$git status
On branch master
nothing to commit, working tree clean
~/Public/A$git log --oneline --graph --decorate --all
* a0f0c77 (HEAD -> master, origin/master) ignore pattern added
* e245bdc gitignore added
* 59a2c83 merged
* dc62f5b (new-login-feature) new-login-1
* b6422a6 new-login-1
* 52a9bd8 second-commit
* ddc0409 first commit
~/Public/A$git branch
* master
  new-login-feature
~/Public/A$git pull origin main
fatal: couldn't find remote ref main
~/Public/A$git pull origin master
From https://github.com/Kanish-Kumar-S/Library
 * branch            master     -> FETCH_HEAD
Already up to date.
~/Public/A$git push origin master
Everything up-to-date
~/Public/A$git tag -a v1.0 -m "Final Project Submission"
~/Public/A$git tag
v1.0
~/Public/A$git push origin v1.0
Enumerating objects: 1, done.
Counting objects: 100% (1/1), done.
Writing objects: 100% (1/1), 186 bytes | 186.00 KiB/s, done.
Total 1 (delta 0), reused 0 (delta 0), pack-reused 0
To https://github.com/Kanish-Kumar-S/Library
 * [new tag]         v1.0 -> v1.0
~/Public/A$
```